

Agri-Chain: Multi-Tier Production Deployment using Docker, Kubernetes, and AWS EKS

Name: Zeenat Parveen

Registration ID: 12324057

Course: Cloud & DevOps (9P188)

Project Title: Agri-Chain: Multi-Tier Production Deployment using Docker, Kubernetes, and AWS EKS

2. Project Overview The Agri-Chain project marks a strategic transition from a legacy monolithic architecture to a modern, cloud-native containerized ecosystem. This evolution addresses the inherent bottlenecks of static server environments by decoupling the compute layer from the delivery mechanism, ensuring that application components are no longer constrained by underlying hardware or tightly coupled dependencies. By moving toward a microservices-oriented design, the system achieves the flexibility required for high-velocity development and deployment. The primary objective was the transformation of the Agri-Chain application—comprising a frontend interface, an AI Crop Doctor backend, and an OTP verification service—into a resilient, production-ready system. This required a rigorous re-architecting of the application's lifecycle to support statelessness and ensure that core services remain performant under varying loads. The result is a system prepared for live-traffic demands, utilizing container orchestration to manage the complexity of its multi-tier components. Leveraging a sophisticated DevOps stack, this project implements industry-standard reliability patterns including managed orchestration and automated delivery pipelines. By establishing a "Code-Ready" infrastructure,

the Agri-Chain system ensures high availability and elasticity. This methodology minimizes human intervention, mitigates the risk of configuration drift, and provides a robust framework for continuous delivery within a scaled AWS environment.

3. DevOps & Cloud Architecture3.1 Containerization Strategy (Docker)

- **Decoupled Microservices:** The architecture separates the frontend and the AI Crop Doctor backend into distinct, lightweight Docker containers. The frontend utilizes **Nginx** for lightning-fast delivery of static assets, while the backend leverages **Node.js** for logic processing.
- **Statelessness and Portability:** By packaging code with its specific runtime and dependencies, we ensure environment parity. This eliminates the "it works on my machine" phenomenon across development, staging, and production.
- **Independent Scalability:** Decoupling allows for granular resource allocation; for example, the AI processing tier can be scaled independently of the web delivery tier during peak agricultural cycles.

3.2 Orchestration

with Kubernetes The deployment is orchestrated using Kubernetes manifests—specifically `frontend-deployment.yaml` and `backend-deployment.yaml`—to manage desired state and self-healing. A **2-replica configuration** was implemented for each service to eliminate Single Points of Failure (SPOF). This redundancy is critical for the Brevo API integration and the OTP verification system, ensuring that user authentication remains uninterrupted. The configuration utilizes Service type: LoadBalancer to bridge the gap between internal pod networking and external user traffic.

3.3 AWS Cloud Infrastructure (EKS)

For the production control plane, the project utilizes **Amazon Elastic Kubernetes Service (EKS)**. The transition from local manifests to the cloud is handled by the AWS Load Balancer Controller, where the LoadBalancer service

type triggers the automated provisioning of an **AWS Application Load Balancer (ALB)**. Verified container images are stored in **Amazon ECR (Elastic Container Registry)**, providing a secure, high-performance private registry that serves as the single source of truth for all cluster deployments.

3.4 Database Strategy (MongoDB) To solve the `MongooseServerSelectionError` and ensure stability, the architecture employs a centralized MongoDB backend managed independently from ephemeral Kubernetes pods. Connectivity is maintained via **environment variables** defined in a `.env` file, which bridges the Node.js backend to the database instance. This approach ensures data persistence across pod restarts and facilitates **Horizontal Pod Autoscaling (HPA) readiness** by keeping the application tier stateless.

4. Automation & CI/CD

4.1 GitHub Actions Workflow The project achieves **GitOps** principles by storing Kubernetes manifests and Dockerfiles alongside application code. The `.github/workflows/cicd.yml` pipeline automates the "Quality Gate" process. Upon every code push, the workflow triggers automated builds and tests, ensuring the infrastructure remains "Code-Ready" for deployment.

4.2 Continuous Integration Benefits

Feature	Manual Process	CI/CD Workflow (Implemented)
Build Process	Manual local builds; inconsistent environments	Standardized builds via GitHub Actions
Deployment Speed	High latency; manual CLI configuration	Automated triggers on every code push
Quality Control	Errors often identified post-deployment	Automated Quality Gates catch errors in build phase
Elasticity	Manual resource scaling	Automated scaling via K8s manifests
Reliability	Prone to human configuration drift	IaC (Infrastructure as Code) consistency

5. Implementation & Local Validation
5.1 Repository Structure The repository is organized to maintain a clear separation between application logic and infrastructure blueprints:

```
agri-chain-cloud-devops/
├── .github/
│   └── workflows/
│       └── cicd.yml                # Automated CI/CD Pipeline
├── backend/
│   ├── Dockerfile                 # Node.js Environment
│   └── Definition
│       ├── package.json           # Dependency Management
│       ├── index.js               # Application Entry Point
│       └── .env                   # Environment/Database
├── Configuration
│   ├── frontend/
│   │   ├── Dockerfile             # Nginx Static Server
│   │   └── Definition
│   │       ├── index.html         # Application UI
│   │       └── kubernetes/
│   │           ├── backend-deployment.yaml # K8s Backend Specs (2
Replicas)
│   │           └── frontend-deployment.yaml # K8s Frontend Specs (2
Replicas)
│   └── docker-compose.yml         # Local Orchestration
└── Blueprint
    └── README.md                  # Technical Documentation
```

5.2 Local Testing Blueprint Pre-cloud validation was conducted using **Docker Compose** to verify the multi-tier interaction. By executing docker-compose up

--build, the system simulated the production environment on localhost. This phase was critical for verifying **WSL (Windows Subsystem for Linux)** integration and ensuring that the Docker Engine correctly handled networking between the Node.js backend and the MongoDB service prior to EKS staging.

6. ConclusionThe Agri-Chain deployment successfully demonstrates a high-maturity DevOps lifecycle. Through the strategic use of Docker for containerization, Kubernetes for managed orchestration, and GitHub Actions for automated delivery, the application has been elevated to production standards. By addressing persistence through centralized data management and ensuring high availability via a multi-replica AWS EKS architecture, the system is fully optimized for live-scaling and resilient performance in a cloud-native environment.